BEYOND WIRELENGTH AND REGULARITY: TIMING-, CONGESTION-AWARE AND MULTI-OBJECTIVE REINFORCEMENT LEARNING FOR MACRO REGULATION

MID-SEMESTER PROJECT REPORT

Pratyush Mohata
2023A3PS0690P
Electrical & Electronics Engg., BITS Pilani

Aditya Birari
2023A3PS0700P
Electrical & Electronics Engg., BITS Pilani

ABSTRACT

Macro placement fixes the positions of large blocks (memories, IP cores) on a chip canvas and strongly determines the final power, performance and area (PPA). Two recent learning-based approaches attack the weaknesses of reinforcement-learning (RL) "placers" that build a layout from an empty canvas. The first is a diffusion model that generates all macro positions at once and is trained only on synthetic data (Lee et al., 2025). The second is *MaskRegulate*, which recasts RL as a *regulator* that refines an existing placement using dense, exact rewards for wirelength and *regularity* (Xue et al., 2024). We review both works and identify three gaps: (i) timing and congestion are measured only after placement and never shape the RL reward, (ii) the wirelength–regularity trade-off is fixed by a single hand-chosen coefficient, and (iii) the regulator's outcome is bounded by the placement it starts from. We propose two extensions. **Direction 1** gives the regulator new information about timing and routing congestion, through a *TimingMask* and a *CongestionMask*, and rewards it for improving both. **Direction 2** replaces the single trade-off with a multi-objective approach: one preference-conditioned regulator that returns a set of placements covering different trade-offs. We also propose starting the regulator from a diffusion-generated placement, which links the two papers. Finally, we describe how these ideas will be evaluated on the ICCAD 2015 benchmarks.

Keywords Macro placement · Reinforcement learning · Diffusion models · Timing-driven placement · Multi-objective RL

1 Introduction

Physical design turns a logical netlist into a manufacturable layout. Its first major step, *placement*, assigns coordinates to every component. Modern chips contain hundreds to thousands of *macros* and millions of small *standard cells*, so placement is usually done in two stages: macros first, then cells around them. Because macros are large and highly connected, their positions largely decide how long the wires are, where routing congestion appears, and whether signals arrive within the clock period (timing).

Google's RL work (Mirhoseini et al., 2021) made macro placement a popular RL benchmark: an agent places macros one at a time on a grid and receives a reward based on wirelength. Follow-up placers (Lai et al., 2022, 2023; Geng et al., 2024) improved representations and sample efficiency, but three problems remain:

- RL placers must be trained per circuit, which takes hours.
- They generalise poorly, because little public data exists.
- They make early, irreversible decisions with incomplete information: the first macro is placed before the agent knows where its neighbours will go.

This project studies two papers that respond to these problems in opposite ways. **Paper A** (Lee et al., 2025) abandons RL and trains a denoising diffusion model that places *all* objects simultaneously and zero-shot. **Paper B**, MaskRegulate

(Xue et al., 2024), keeps RL but changes its job. The policy *refines* an existing placement instead of building one from scratch, and a *regularity* objective pushes macros towards the canvas periphery.

Contributions of this report.

1. A review of both papers and a gap analysis based on the papers' own reported results (Section 3).
2. **Direction 1:** a timing- and congestion-aware regulator with two new input masks and reward terms (Section 4.2).
3. **Direction 2:** a multi-objective regulator that produces a set of trade-off placements from a single trained policy (Section 4.3).
4. A plan for evaluating both directions (Section 5).

2 Background

Netlist and placement. A netlist is a hypergraph $\mathcal{H} = (V, E)$. Each net $e \in E$ connects a set of pins P_e on objects in $V = V_{\text{mac}} \cup V_{\text{cell}} \cup V_{\text{io}}$. Macro i has width w_i and height h_i. A macro placement $s = \{(x_i, y_i)\}_{i=1}^{N}$ gives the lower-left corner of each of the N macros. It is *legal* if macros lie inside the canvas $[0, X_{\max}] \times [0, Y_{\max}]$ and their rectangles do not overlap.

Half-perimeter wirelength (HPWL).

$$\text{HPWL}(s) = \sum_{e \in E} \Big(\max_{p \in P_e} x_p - \min_{p \in P_e} x_p \Big) + \Big(\max_{p \in P_e} y_p - \min_{p \in P_e} y_p \Big). \tag{1}$$

It is cheap to compute and is the main wirelength proxy in both papers.

Regularity (Xue et al., 2024). The regularity of macro i is the sum of its distances to the nearest vertical and horizontal canvas edges:

$$\text{Reg}(s) = \sum_{i=1}^{N} \Big[\min\{x_i,\, X_{\max} - x_i - w_i\} + \min\{y_i,\, Y_{\max} - y_i - h_i\} \Big]. \tag{2}$$

It is zero for a macro in a corner and largest at the centre, so lower is better.

Timing. A static timing analysis (STA) tool computes when each signal arrives and compares it with the time it is required, which is set by the clock period T. The difference is the *slack*; a negative slack means the signal is late. Two standard summaries are used: the *worst negative slack* (WNS), which is the single latest path, and the *total negative slack* (TNS), which is the sum over all late paths. For both, values closer to zero are better.

Reinforcement learning. An agent observes a *state*, takes an *action* and receives a *reward*. It learns a *policy* that maximises the total reward over an episode. Both MaskRegulate and our extensions train the policy with PPO (Schulman et al., 2017), a standard and stable policy-gradient algorithm.

3 Literature Review

3.1 Classical and learning-based macro placement

Analytical placers such as DREAMPlace (Lin et al., 2021) relax placement into a smooth problem (wirelength plus a density penalty) and solve it with GPU gradient descent. They are fast, but they tend to cluster macros in the centre and can stall in local optima. AutoDMP (Agnesina et al., 2023) tunes DREAMPlace's settings with Bayesian optimisation, and DREAMPlace 4.0 (Liao et al., 2023) makes it timing-driven by re-weighting critical nets. *Black-box* methods such as WireMask-BBO (Shi et al., 2023) search placements guided by a wire mask. *RL placers*, including GraphPlace (Mirhoseini et al., 2021), MaskPlace (Lai et al., 2022), ChiPFormer (Lai et al., 2023) and EfficientPlace (Geng et al., 2024), place macros sequentially on a grid. Independent assessments have questioned how well RL placers' proxy gains carry over to real PPA (Cheng et al., 2023).

3.2 Paper A: Chip Placement with Diffusion Models (Lee et al., 2025)

Objective. Train a model once, offline, that can then place the macros of *any* new chip in a few minutes, without retraining it for that chip.

Main idea. The paper uses a *diffusion model*, the same kind of generative model used by modern image generators. Generation starts from macros scattered at random positions. The model then nudges all of them a little at a time, over many small steps, until a clean, compact layout emerges. Because every macro moves together, no early decision is locked in, unlike in sequential RL placers.

Training data. Real chip designs are confidential, so there is very little data to learn from. The authors solve this by running the problem in reverse: they first scatter blocks on a canvas and only then add wires, mostly between blocks that are already close together. The layout they started from is therefore a good placement for the circuit they generated. This gives an unlimited supply of (circuit, good placement) examples at almost no cost.

Guidance. While generating a layout, the model is gently steered towards shorter wires and away from overlapping macros, without any retraining.

Reported results. On standard academic benchmarks (IBM and ISPD 2005), the paper reports:

- placements in minutes per chip, instead of the hours needed by RL methods;
- substantially shorter wirelength and less congestion than earlier RL and search-based placers;
- results slightly better than DREAMPlace, a widely used analytical placer.

Shortcomings.

- It only measures quick estimates (wirelength, congestion). Timing is never checked.
- It cannot easily trade extra compute for better quality. The authors suggest RL fine-tuning as future work.
- Overlaps are discouraged but not strictly prevented.

3.3 Paper B: MaskRegulate (Xue et al., 2024)

Objective. Use RL to *refine* a placement produced by DREAMPlace rather than create one, and optimise for real PPA rather than HPWL alone.

MDP.

- An episode visits each macro once, in a rule-based order.
- The state is a set of 224×224 image masks for the current macro and the next one:
  - The *PositionMask* marks feasible cells; only already-adjusted macros count as obstacles.
  - The *WireMask* gives the exact change in HPWL for each cell, normalised to $[-1, 1]$.
  - The *RegularMask* gives the change in regularity.
- The action chooses one of 224^2 cells for the current macro.
- The reward is $r = \alpha\, r_{\text{wire}} + (1 - \alpha)\, r_{\text{reg}}$, where both terms are normalised to $[0, 1]$ and $\alpha = 0.7$.

The policy fuses masks pixel-wise with 1×1 convolutions and combines them with a ResNet-18 encoder/decoder, before masking infeasible cells and applying a softmax. Training uses PPO. Because every other macro already has a position, the effect of each move is computed exactly. This removes the incomplete-information and sparse-reward problems of placers.

Reported results. On the ICCAD 2015 *superblue* circuits (Kim et al., 2015), evaluated with Cadence Innovus, MaskRegulate has the best average rank on all eight metrics. Compared with MaskPlace, it improves routed wirelength by 17.08%, horizontal and vertical overflow by 73.08% and 38.81%, WNS by 18.35%, TNS by 37.89%, and NVP by 46.17%. It also generalises to unseen chips and can refine the outputs of other placers.

Shortcomings.

- *Timing and congestion are never in the reward.* The authors list overlooking "global wirelength and timing metrics during the training process" as a limitation. Their results agree:
  - The average WNS and TNS ranks are both 1.875, against 1.0–1.125 for regularity, routed wirelength and horizontal overflow.

Table 1: Side-by-side comparison of the two reviewed papers.

	Paper A: Diffusion (Lee et al., 2025)	**Paper B: MaskRegulate** (Xue et al., 2024)
View of RL	Replace it	Keep it, but as a refiner
Starting point	Random positions	An existing (DREAMPlace) placement
Decision style	All macros at once, 1,000 denoising steps	One macro per step, one pass
Learning	Generative (denoising) on synthetic data	PPO on real benchmark circuits
Representation	Graph (GNN + attention), continuous	Image masks (224^2 grid), CNN
Objectives	HPWL + overlap via guidance	HPWL + regularity via reward
Legality	Soft penalty	Hard (PositionMask)
Evaluation	Proxy only (HPWL, RUDY)	Proxy + Innovus routing/timing
Strength	Zero-shot, minutes per circuit	Best PPA ranks, accurate dense reward
Main gap	No PPA-aware optimisation, no RL fine-tuning	No timing/congestion in reward; single α

  - AutoDMP has better WNS on superblue4, 16 and 18.
  - On superblue7, MaskRegulate's vertical overflow (0.64%) is about double that of MaskPlace (0.33%) and WireMask-EA (0.35%).
- *A single trade-off.* α is fixed by hand. The authors' sweep shows different α give different trade-offs, and they name multi-objective optimisation for different chip preferences as future work.
- *Dependence on the start.* The regulator edits locally, so a poor initial placement bounds the outcome.
- The grid is discrete, training takes about 1,000 PPO episodes per chip, and macro aspect ratio is ignored.

3.4 Comparison and research gaps

Table 1 summarises the two papers. They are complementary: Paper A is a fast, zero-shot *drafter* with no notion of PPA beyond HPWL, while Paper B is an accurate *editor* that needs a draft and optimises only two proxies. Concurrent work, FlowPlace (Xie et al., 2026), folds a periphery prior into the synthetic data of a flow-matching generator. It uses neither RL fine-tuning nor multi-objective selection. We target three gaps:

G1 Timing and congestion should enter the regulator's *state and reward*, not only its evaluation. This is Direction 1.

G2 Designers need a *set* of trade-off placements, not one α. This is Direction 2.

G3 The regulator's outcome is bounded by its start, which motivates a diffusion-generated start (Section 4.4).

4 Proposed Methodology

We keep MaskRegulate's overall framework and extend it in two directions. Figure 1 shows the complete flow:

1. An initial macro placement is produced, either by DREAMPlace (as in MaskRegulate) or by the diffusion model of Paper A.
2. An RL *regulator* refines this placement one macro at a time.
3. Standard cells are placed around the fixed macros with DREAMPlace.
4. The final layout is routed and its timing is checked.

Our changes are all inside step 2: what the regulator *sees* (its input masks) and what it is *rewarded* for.

4.1 How the regulator works

The chip canvas is divided into a grid of cells. Macros are visited one at a time, largest first. In each episode, every macro is visited exactly once:

- **What the agent sees (state).** A set of grid-shaped "masks" for the current macro. Each mask answers one question for every grid cell, for example: "Is this spot free?" (PositionMask) or "How would wirelength change if the macro moved here?" (WireMask).
- **What the agent does (action).** It picks a free grid cell and moves the current macro there.

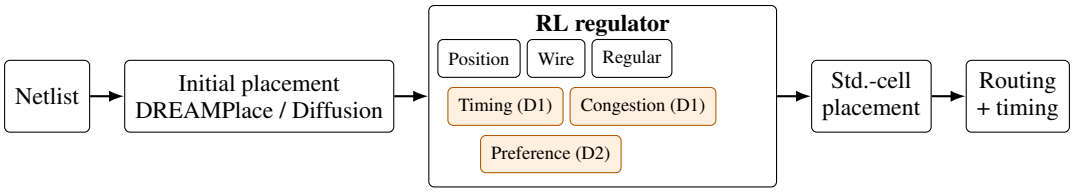


Figure 1: Proposed flow. White boxes are taken from MaskRegulate. Orange boxes are new in this project: timing and congestion information (Direction 1, D1) and a designer preference input (Direction 2, D2).

- **What the agent gets (reward).** The improvement that the move produced in each objective, combined with weights:

$$r = w_{\text{wire}}\, r_{\text{wire}} + w_{\text{reg}}\, r_{\text{reg}} + w_{\text{tim}}\, r_{\text{tim}} + w_{\text{cong}}\, r_{\text{cong}}. \tag{3}$$

  MaskRegulate uses only the first two terms ($w_{\text{wire}} = 0.7$, $w_{\text{reg}} = 0.3$). The timing and congestion terms are our additions.

Each reward term is scaled by the objective's starting value so that terms with different units are comparable. The per-move rewards also add up exactly to the total improvement over the whole episode. This means the agent receives frequent, accurate feedback, and it is still being trained to optimise the *final* layout. It also means that a new objective can be added to the regulator as long as we can compute how each possible move would change it. That is exactly what the two new masks below provide.

The policy network (a CNN) and the training algorithm (PPO) are kept as in MaskRegulate. Only the input masks and the reward change.

4.2 Direction 1: timing- and congestion-aware regulation

Motivation. MaskRegulate pushes macros towards the chip edges (regularity). This clears the centre for routing, but it can stretch the wires on timing-critical paths that pass through those macros. Its own results show timing as its weakest metric. We therefore give the regulator direct information about timing and congestion.

4.2.1 TimingMask

- **Delay model.** Wire delay grows faster than wire length. For a simple driver-and-wire model (the Elmore delay, Elmore, 1948), it has a constant part, a part proportional to the length ℓ, and a part proportional to ℓ^2:

$$d(\ell) \approx d_0 + a\,\ell + b\,\ell^2. \tag{4}$$

  A long wire is therefore disproportionately slow. Moving a macro to the edge can create exactly such long wires.
- **Which paths matter.** A static timing analysis (STA) tool (OpenSTA or OpenTimer) lists the most critical paths and their delays. The STA is re-run every few training episodes, not after every move, to keep it affordable.
- **Timing cost.** We add up how much each critical path exceeds the clock period T. A small safety margin also counts paths that are *nearly* late:

$$\text{Timing cost} = \sum_{\text{critical paths}} \max\big(0,\ \text{path delay} - 0.95\,T\big). \tag{5}$$

- **The mask.** Moving a macro only changes the two wires connecting it to its neighbours on each path. So for every grid cell we can compute exactly how the timing cost would change. This produces the TimingMask, which works the same way as MaskRegulate's WireMask.
- **Cheaper alternative.** Instead of tracking whole paths, give more weight to nets that lie on critical paths inside the existing WireMask. This is similar to how DREAMPlace 4.0 makes placement timing-driven (Liao et al., 2023). We will compare both versions.

4.2.2 CongestionMask

- **Routing demand.** RUDY (Spindler and Johannes, 2007) estimates how many wires pass through each region of the chip by spreading each net's wire evenly over its bounding box. Total demand over the whole chip equals the total wirelength. Wirelength alone therefore controls how *much* demand there is, but not *where* it concentrates.

- **Routing capacity.** Each region can carry a fixed number of wires. A macro blocks part of the routing resources above it, so a region covered by a macro has less capacity.
- **Overflow.** Congestion is measured as the demand that exceeds capacity:

$$\text{Overflow} = \sum_{\text{regions}} \max\big(0,\ \text{demand} - \text{capacity}\big). \tag{6}$$

- **The mask.** For every grid cell, the CongestionMask gives the extra overflow that would be created if the macro were placed there, i.e. if it blocked a busy routing region. This can be computed for the whole grid in one fast pass.

Regularity is an indirect rule ("keep macros near the edge"). The CongestionMask measures directly what that rule tries to protect ("do not block busy routing regions"), and only where it actually matters.

4.2.3 Putting it together

The regulator now receives five masks: Position, Wire and Regular from MaskRegulate, plus Timing and Congestion. Its reward is Eq. (3) with all four terms switched on. Setting the timing and congestion weights to zero recovers MaskRegulate exactly, which gives us a clean baseline.

4.3 Direction 2: multi-objective regulation

Motivation. Different chips need different trade-offs. A mobile chip may prioritise wirelength and power, while a high-performance chip prioritises timing. MaskRegulate fixes one trade-off ($\alpha = 0.7$) by hand and returns a single placement. Its authors list multi-objective optimisation as future work.

Goal: a menu of good placements. Instead of one answer, we want to give the designer a *set* of placements, each representing a different trade-off. A placement is *Pareto-optimal* if no other placement is better in one objective without being worse in another. The set of such placements is the *Pareto front*, and that is what we want the regulator to produce.

Approach.

1. **Baseline: one regulator per trade-off.** Train separate regulators for several fixed weight settings $(w_{\text{wire}}, w_{\text{reg}}, w_{\text{tim}}, w_{\text{cong}})$. This generalises MaskRegulate's own α sweep, but the training cost grows with the number of settings.
2. **Proposed: one preference-conditioned regulator.** Train a *single* policy that takes the preference weights as an extra input. During training, a new random preference is drawn for every batch of episodes, and the reward uses those weights. At test time, the designer chooses the weights, or we sweep over many of them, and the same trained model returns the matching placement. This follows the idea of preference-driven multi-objective RL (Basaklar et al., 2023). The preference is injected into the network with a simple conditioning layer (FiLM, Perez et al., 2018).
3. **Reaching every trade-off.** A weighted sum of objectives can miss some balanced trade-offs. As a refinement, we will also try the *Chebyshev* rule, which scores a layout by its worst-weighted objective. Because the final objective values are known at the end of each episode, this can be given as a final-step reward.

How we will judge a set of placements. We use the standard *hypervolume* measure (Zitzler and Thiele, 1999). It is the size of the region of objective space that the set improves upon, relative to a fixed reference point. A set scores higher if its placements are both better and more varied. We will evaluate on 35 evenly spread preference settings, including the four "single-objective" extremes (Das and Dennis, 1998).

4.4 Linking the two papers: a better starting point

The regulator only edits a placement, so a better draft should lead to a better final layout. We will therefore replace DREAMPlace's starting placement with one sampled from the pre-trained diffusion model of Paper A. This needs no extra training. As a stretch goal, the diffusion model itself could be fine-tuned with RL (DDPO, Black et al., 2024), with the regulator's final improvement as the reward. This is the future work both papers point towards.

Table 2: Summary of the proposed components.

Component	What it does	Gap addressed
TimingMask	Shows, for every grid cell, how the delay of critical paths would change	G1: timing not in the reward
CongestionMask	Shows, for every grid cell, the extra routing overflow caused by blocking	G1: congestion not in the reward
Four-term reward	Rewards improvements in wirelength, regularity, timing and congestion	G1
Preference input	Lets one trained policy produce placements for any trade-off	G2: single fixed α
Hypervolume evaluation	Scores the quality and variety of the returned set	G2
Diffusion start	Gives the regulator a better draft to edit	G3: dependence on the start

5 Planned Experimental Protocol

Benchmarks. Our main benchmark is the ICCAD 2015 superblue suite (Kim et al., 2015): eight industrial-scale designs with 0.8–1.9M cells, hundreds of macros and timing constraints. We will also use ISPD 2005 for comparison with Paper A. Small synthetic netlists, generated with Paper A's method, will be used for quick debugging.

Baselines. DREAMPlace (Lin et al., 2021), AutoDMP (Agnesina et al., 2023), MaskPlace (Lai et al., 2022), MaskRegulate (Xue et al., 2024) (official code) and the pre-trained diffusion placer (Lee et al., 2025).

Evaluation flow and metrics.

- Each method places the macros. Standard cells are then placed by DREAMPlace with the macros fixed.
- Routing and timing analysis use Cadence Innovus if a licence is available, and the open-source OpenROAD flow (Ajayi et al., 2019) otherwise.
- *Quick (proxy) metrics*: wirelength (HPWL), regularity, estimated congestion.
- *Final (PPA) metrics*: routed wirelength, routing overflow, WNS, TNS and the number of violating paths.
- *Set metrics* (Direction 2): hypervolume and the number of Pareto-optimal placements.

Planned experiments. Each experiment in Table 3 tests one hypothesis.

Table 3: Planned experiments. "sb" denotes superblue.

ID	Question	Setup	Measured by
E1	Can we reproduce MaskRegulate's published numbers?	Official code on sb1/3/4/5	HPWL and regularity within 5%
E2	Does the TimingMask improve timing, and at what wirelength cost?	MaskRegulate vs. + timing term; full vs. cheaper variant	WNS, TNS, violating paths, routed WL
E3	Does the CongestionMask reduce routing overflow beyond regularity?	MaskRegulate vs. + congestion term	Routing overflow, routed WL
E4	Do timing and congestion pull in different directions?	Several weight settings	Per-metric comparison
E5	Can one preference-conditioned policy replace many separately trained ones?	35 preference settings	Hypervolume, training time
E6	Does a diffusion starting point improve the final layout?	Start from DREAMPlace / diffusion / random	All final metrics
E7	Does the regulator work on chips it has not seen?	Train on sb1/3/4/5, test on sb7/10/16/18	Final metrics on test chips

Expected challenges.

- *Compute*: MaskRegulate was trained on four GPUs. We will start from released checkpoints and the smaller superblue designs.

- *Tool access*: without an Innovus licence we will use the OpenROAD flow and compare methods relative to each other.
- *Timing analysis cost*: if path-level timing is too slow inside training, we will use the cheaper critical-net weighting.
- *Balancing objectives*: the reward weights decide which objective dominates. Direction 2 addresses this by learning across many weightings.

6 Conclusion

The diffusion placer (Lee et al., 2025) is a fast, zero-shot drafter, and MaskRegulate (Xue et al., 2024) is an accurate RL editor. Neither lets timing or congestion guide its optimisation, and MaskRegulate commits to a single hand-chosen trade-off. We propose two extensions:

- **Direction 1:** give the regulator two new inputs and reward terms, a *TimingMask* for critical-path delay and a *CongestionMask* for routing overflow caused by macro blockage.
- **Direction 2:** train one preference-conditioned regulator that returns a set of placements covering different trade-offs, judged by hypervolume.

We will also test whether starting the regulator from a diffusion-generated placement improves the final layout. The remaining part of the semester will be spent implementing these components and running the planned experiments on the ICCAD 2015 benchmarks.

References

Agnesina, A., Rajvanshi, P., Yang, T., Pradipta, G., Jiao, A., Keller, B., Khailany, B., and Ren, H. (2023). AutoDMP: Automated DREAMPlace-based macro placement. In *Proceedings of the International Symposium on Physical Design (ISPD)*, pages 149–157.

Ajayi, T., Chhabria, V. A., Fogaça, M., Hashemi, S., Hosny, A., Kahng, A. B., Kim, M., Lee, J., Mallappa, U., Neseem, M., et al. (2019). Toward an open-source digital flow: First learnings from the OpenROAD project. In *Proceedings of the 56th Design Automation Conference (DAC)*.

Basaklar, T., Gumussoy, S., and Ogras, U. Y. (2023). PD-MORL: Preference-driven multi-objective reinforcement learning algorithm. In *International Conference on Learning Representations (ICLR)*.

Black, K., Janner, M., Du, Y., Kostrikov, I., and Levine, S. (2024). Training diffusion models with reinforcement learning. In *International Conference on Learning Representations (ICLR)*.

Cheng, C.-K., Kahng, A. B., Kundu, S., Wang, Y., and Wang, Z. (2023). Assessment of reinforcement learning for macro placement. In *Proceedings of the International Symposium on Physical Design (ISPD)*.

Das, I. and Dennis, J. E. (1998). Normal-boundary intersection: A new method for generating the Pareto surface in nonlinear multicriteria optimization problems. *SIAM Journal on Optimization*, 8(3):631–657.

Elmore, W. C. (1948). The transient response of damped linear networks with particular regard to wideband amplifiers. *Journal of Applied Physics*, 19(1):55–63.

Geng, Z., Wang, J., Liu, Z., Xu, S., Tang, Z., Yuan, M., Hao, J., Zhang, Y., and Wu, F. (2024). Reinforcement learning within tree search for fast macro placement. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*.

Kim, M.-C., Hu, J., Li, J., and Viswanathan, N. (2015). ICCAD-2015 CAD contest in incremental timing-driven placement and benchmark suite. In *IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*.

Lai, Y., Liu, J., Tang, Z., Wang, B., Hao, J., and Luo, P. (2023). Chipformer: Transferable chip placement via offline decision transformer. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*.

Lai, Y., Mu, Y., and Luo, P. (2022). Maskplace: Fast chip placement via reinforced visual representation learning. In *Advances in Neural Information Processing Systems 35 (NeurIPS)*.

Lee, V., Nguyen, M., Elzeiny, L., Deng, C., Abbeel, P., and Wawrzynek, J. (2025). Chip placement with diffusion models. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, volume 267 of *PMLR*, pages 33499–33514.

Liao, P., Guo, D., Guo, Z., Liu, S., Lin, Y., and Yu, B. (2023). DREAMPlace 4.0: Timing-driven placement with momentum-based net weighting and Lagrangian-based refinement. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 42(10):3374–3387.

Lin, Y., Jiang, Z., Gu, J., Li, W., Dhar, S., Ren, H., Khailany, B., and Pan, D. Z. (2021). DREAMPlace: Deep learning toolkit-enabled GPU acceleration for modern VLSI placement. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 40(4):748–761.

Mirhoseini, A., Goldie, A., Yazgan, M., Jiang, J. W., Songhori, E., Wang, S., Lee, Y.-J., Johnson, E., Pathak, O., Nazi, A., et al. (2021). A graph placement methodology for fast chip design. *Nature*, 594(7862):207–212.

Perez, E., Strub, F., de Vries, H., Dumoulin, V., and Courville, A. (2018). FiLM: Visual reasoning with a general conditioning layer. In *Proceedings of the AAAI Conference on Artificial Intelligence*.

Schulman, J., Wolski, F., Dhariwal, P., Radford, A., and Klimov, O. (2017). Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*.

Shi, Y., Xue, K., Song, L., and Qian, C. (2023). Macro placement by wire-mask-guided black-box optimization. In *Advances in Neural Information Processing Systems 36 (NeurIPS)*.

Spindler, P. and Johannes, F. M. (2007). Fast and accurate routing demand estimation for efficient routability-driven placement. In *Design, Automation and Test in Europe (DATE)*.

Xie, P., Xue, K., Shi, Y., Chen, R.-T., Gao, C., Xu, S., Ding, C., Yuan, M., and Qian, C. (2026). FlowPlace: Flow matching for chip placement. In *Proceedings of the 63rd ACM/IEEE Design Automation Conference (DAC)*. arXiv:2604.23658.

Xue, K., Chen, R.-T., Lin, X., Shi, Y., Kai, S., Xu, S., and Qian, C. (2024). Reinforcement learning policy as macro regulator rather than macro placer. In *Advances in Neural Information Processing Systems 37 (NeurIPS)*.

Zitzler, E. and Thiele, L. (1999). Multiobjective evolutionary algorithms: A comparative case study and the strength Pareto approach. *IEEE Transactions on Evolutionary Computation*, 3(4):257–271.